Split Packages

Jave Platform Module System

Example

As an example, let's fiddle with the *ServiceMonitor* application. As you may recall, the *monitor.statistics* module contains a package `monitor.statistics`. Let's create a package with the same name (and the class `SimpleStatistician`) in *monitor*. When compiling that module, I get the following error:

```
> monitor/src/main/java/monitor/statistics/SimpleStatistician.java:1:
>     error: package exists in another module: monitor.statistics
>         package monitor.statistics;
>         ^
> 1 error
```

Problem

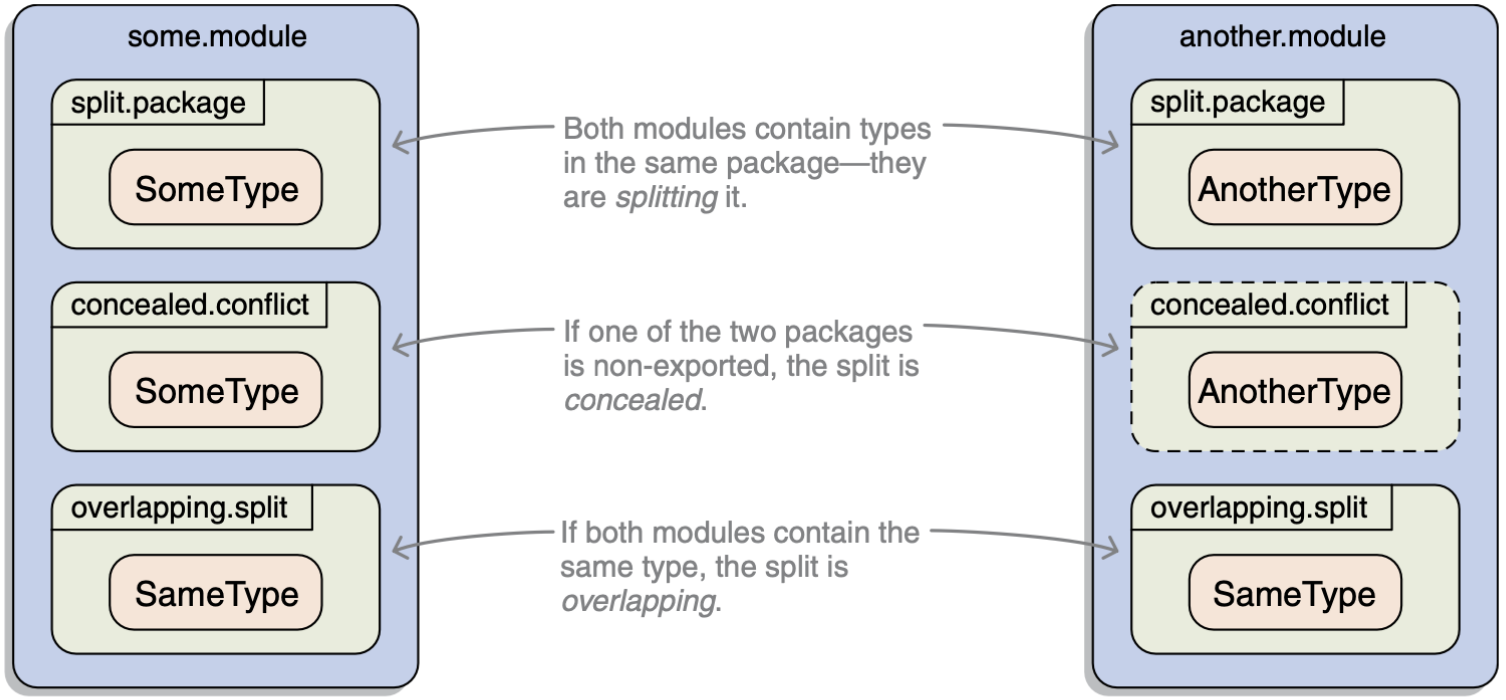


Figure 7.2 When two modules contain types in the same package, they split the package.

Handle Split Packages

There are quite a few ways to make a split package work. Here they are, in the general order I recommend considering them:

- Rename one of the packages.
- Move all parts of the split package into the same artifact.
- Merge the artifacts.

Finding split packages

After copying all dependencies into the `lib` folder, you can execute JDeps as follows:

```
$ jdeps -summary
    -recursive --class-path 'libs/*' project.jar

> split package: javax.annotation
>     [jrt:/java.xml.ws.annotation, libs/jsr305-3.0.2.jar]
>
# lots of project dependencies truncated
```

That's unambiguous, right? If you're curious what depends on the split package, you can use `--package` and `-verbose:class`:

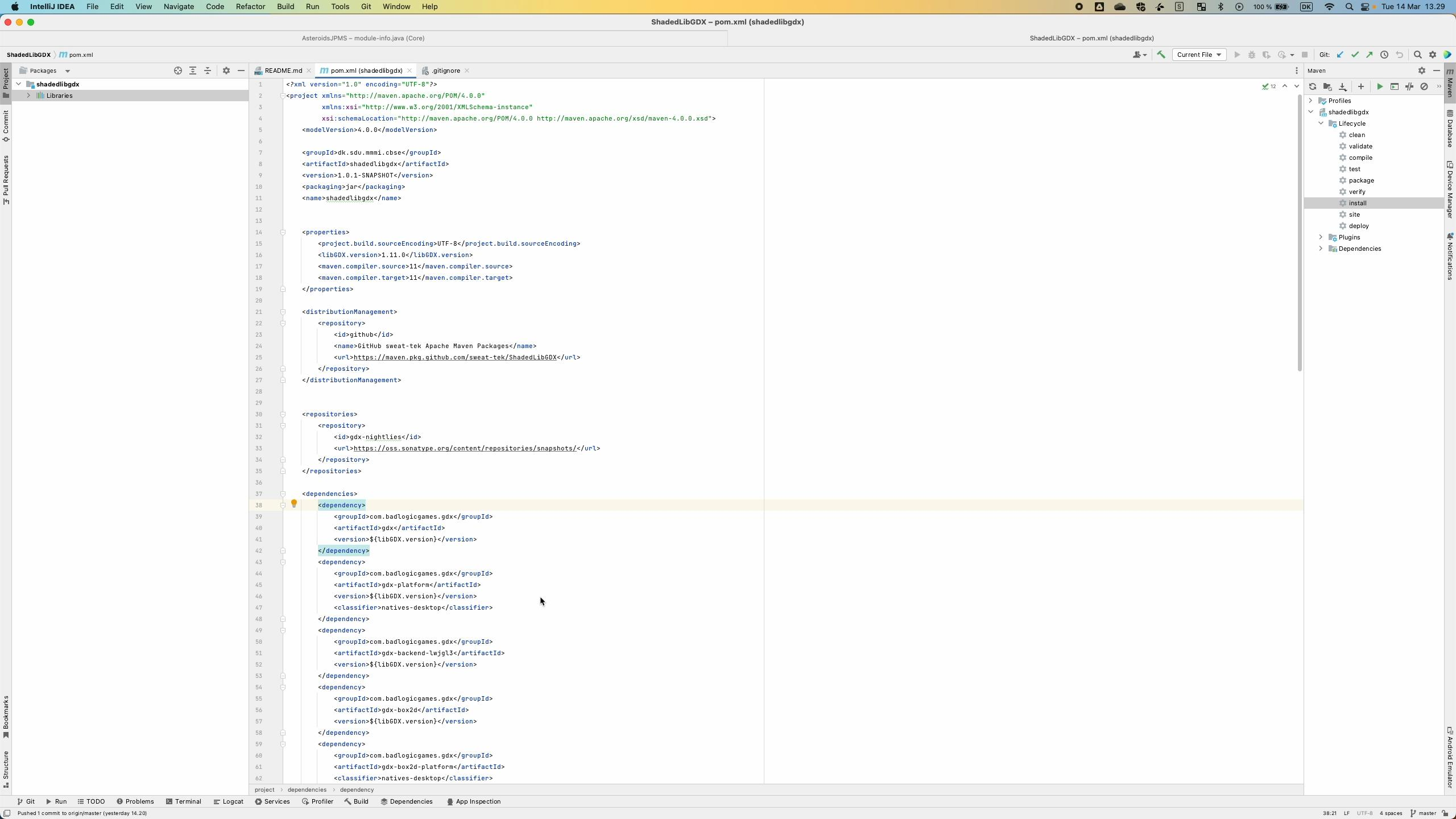
```
$ jdeps -verbose:class
    --package javax.annotation
    -recursive --class-path 'libs/*' project.jar

# split packages truncated
# dependencies *from* javax.annotation truncated

> rest-1.0-SNAPSHOT.jar -> libs/jsr305-3.0.2.jar
>     monitor.rest.MonitorServer -> Nonnull jsr305-3.0.2.jar
```

Merge Artifacts

- maven-shade-plugin
- moditect-maven-plugin


IntelliJ IDEA File Edit View Navigate Code Refactor Build Run Tools Git Window Help
Tue 14 Mar 13.29
ShadedLibGDX – pom.xml (shadedlibgdx)
AsteroidsJPMS – module-info.java (Core)
ShadedLibGDX – pom.xml (shadedlibgdx)
ShadedLibGDX pom.xml
Current File
Git:
Packages
shadedlibgdx
Libraries
README.md
pom.xml (shadedlibgdx)
.gitignore
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
<groupId>dk.sdu.mmmi.cbse</groupId>
<artifactId>shadedlibgdx</artifactId>
<version>1.0.1-SNAPSHOT</version>
<packaging>jar</packaging>
<name>shadedlibgdx</name>
<properties>
<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
<libGDX.version>1.11.0</libGDX.version>
<maven.compiler.source>11</maven.compiler.source>
<maven.compiler.target>11</maven.compiler.target>
</properties>
<distributionManagement>
<repository>
<id>github</id>
<name>GitHub sweat-tek Apache Maven Packages</name>
<url>https://maven.pkg.github.com/sweat-tek/ShadedLibGDX</url>
</repository>
</distributionManagement>
<repositories>
<repository>
<id>gdx-nightlies</id>
<url>https://oss.sonatype.org/content/repositories/snapshots/</url>
</repository>
</repositories>
<dependencies>
<dependency>
<groupId>com.badlogicgames.gdx</groupId>
<artifactId>gdx</artifactId>
<version>${libGDX.version}</version>
</dependency>
<dependency>
<groupId>com.badlogicgames.gdx</groupId>
<artifactId>gdx-platform</artifactId>
<version>${libGDX.version}</version>
<classifier>natives-desktop</classifier>
</dependency>
<dependency>
<groupId>com.badlogicgames.gdx</groupId>
<artifactId>gdx-backend-lwjgl3</artifactId>
<version>${libGDX.version}</version>
</dependency>
<dependency>
<groupId>com.badlogicgames.gdx</groupId>
<artifactId>gdx-box2d</artifactId>
<version>${libGDX.version}</version>
</dependency>
<dependency>
<groupId>com.badlogicgames.gdx</groupId>
<artifactId>gdx-box2d-platform</artifactId>
<classifier>natives-desktop</classifier>
project dependencies dependency
Maven
Profiles
shadedlibgdx
Lifecycle
clean
validate
compile
test
package
verify
install
site
deploy
Plugins
Dependencies
Git Run TODO Problems Terminal Logcat Services Profiler Build Dependencies App Inspection
Pushed 1 commit to origin/master (yesterday 14.20)
38:21 LF UTF-8 4 spaces master

Summary

- The module system forbids two modules (in the same layer) to contain the same package—exported or not. This isn't checked for code on the class path, though, so an undiscovered package split between a platform module and class-path code is possible.
- If a package is split between a module and the class path, the class-path portion is essentially invisible, leading to surprising compile-time and run-time errors. The best fix is to remove the split, but if that isn't possible, the platform module in question can either be replaced with the splitting artifact with `--upgrade-module-path` (if it's an upgradeable module) or patched with its content with `--patch-module`.